ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE

School of Computer and Communication Sciences

Handout 21 Principles of Digital Communications
Project Description Apr. 29, 2026

Channel Model.

We wish to communicate over a channel which exhibits one of four possible behaviors. Given a two dimensional vector $x = (x_1, x_2) \in \mathbb{R}^2$, the output $Y \in \mathbb{R}^2$ is related to x as $Y = T_i(x) + Z$, where $Z \sim \mathcal{N}(0, I_2)$ is i.i.d. Gaussian noise and T_i is one of

$$T_1 : x \to x, \quad T_2 : (x_1, x_2) \to (-x_2, x_1), \quad T_3 : x \to -x, \quad T_4 : x \to -T_2(x). \tag{1}$$

If we think of $x = (x_1, x_2)$ as the complex number $x_1 + jx_2$, the operations are multiplications by 1, j, -1 and $-j$.

For an even positive integer n, channel input $x \in \mathbb{R}^n$, and an operation $T = T_i$, we define

$$T(x) = \big(T(x_1, x_2), T(x_3, x_4), \ldots, T(x_{n-1}, x_n)\big), \tag{2}$$

e.g., $T_2(1, 2, 3, 4) = (-2, 1, -4, 3)$. The channel output $Y \in \mathbb{R}^n$ is then

$$Y = T(x) + Z, \tag{3}$$

where $Z_1, \cdots, Z_n$ are i.i.d. $\mathcal{N}(0, 1)$, i.e., i.i.d. Gaussian noise with unit variance. Although the operation T_i is fixed once transmission begins, it will be unknown to you a priori to transmission, i.e., each will occur with equal probability.

Assignment.

Develop a system capable of transmitting short text messages over the above channel in a reliable and efficient manner. Specifically:

- Design a transmitter that reads a 40 character string of 6-bit ASCII characters (totaling 240 bits) and returns real-valued samples of an information-bearing signal $c_i \in \mathbb{R}^n$. The 6-bit character set consists of

  $$\mathcal{A} = \{\texttt{'a'}, \ldots, \texttt{'z'}, \texttt{'A'}, \ldots, \texttt{'Z'}, \texttt{'0'}, \texttt{'1'}, \ldots, \texttt{'9'}, \texttt{' '}, \texttt{'.'}\},$$

  which has 64 elements.

  The block length n of your design is to be even and $n \leq 500$. We also enforce a constraint on the transmit energy to be $\|x\|^2 \leq 1200$. The channel will reject inputs that violate these constraints.

- Send c_i to a server (which we provide, see below) that applies the channel effect as in (3) and returns $Y \in \mathbb{R}^n$.

- Design a receiver that, having received Y, reconstructs the original text message with as few errors as possible.

Hint: Solve the theory part first.

Submission and Evaluation Rules.

- The project is to be done in teams of *four* (but we can also allow a small number of teams of *three*). Please choose your teammates at the latest by **Wednesday, May 6** and send an email to ryan.song@epfl.ch with the subject "PDC project team" in order to register your team.
- For the theoretical part, please prepare one solution per team, and submit it through Moodle. The deadline for this submission is on **Wednesday, May 27**.
- For the practical part, you may use any programming language, as long as all the code pertaining to the transmitter and receiver is produced by your team.
- During the last session of the semester (**May 29**), each group presents their project in 5–10 minutes and demonstrates their working implementation using a text message that we provide.
  (i) You will run the transmitter and the receiver on your own laptop.
  (ii) You must submit your team's code to Moodle before **Friday, May 29, 1 pm**. One submission per team is enough.
  (iii) Your presentation should contain a brief explanation of your signaling scheme (this is not expected to be formal; there is no need to prepare slides, we simply want you to be able to explain your choices in designing your transmitter and receiver), followed by the transmission and decoding of the chosen text (that will be given to you on the spot).
  (iv) You will be given *two* chances for transmission, i.e., if you fail to transmit the message during your first transmission, you can repeat the transmission once more.
- The grade is based on the solution that you submit for the theoretical part, and the reliability of your scheme during the demonstration. The theoretical part counts for 7 points of the grade and the demonstration part counts for 13 points (together counting for a total of 20 points for the project). The demonstration part will be graded as follows:
  (a) If you manage to transmit the text message without error during one of the two transmissions, you will get full marks (13 pts).
  (b) Otherwise your score will be $\max\{10-\chi, 0\}$ where χ is the number of incorrect characters in the reproduced text at the receiver. We will grade based on the better of the two transmissions.
  (c) Additionally, the team which uses smallest energy $\|x\|^2$ (with $\chi = 0$) will get 5 bonus points.
  (d) You will be given 0 if you do not attend the presentation and your teammates cannot specify your contribution during the presentation.

Python Client.

During evaluation on May 29th, we will simulate the channel (3) by a server, to which you can send an input $x \in \mathbb{R}^n$ and receive the noisy output $Y \in \mathbb{R}^n$. The server will be available for testing a few days before the evaluation day. During the development process, we suggest that each team test their implementation locally using the code snippit provided on the next page.

To access this server, we provide you a Python script `client.py` that you can download from Moodle. Please read the associated docstrings for more information. You can only connect to the server if you are inside the EPFL network. If you are working outside of EPFL, you can use EPFL's VPN (please visit the link[1] for more information). Please use the following connection parameters:

- `--srv_hostname=iscsrv72.epfl.ch`
- `--srv_port=80`

For example, running the command

```
python3 client.py --input_file input.txt --output_file output.txt
--srv_hostname=iscsrv72.epfl.ch --srv_port=80
```

reads x from `input.txt` and writes the corresponding Y to `output.txt`.

[You may need to call `python` instead of `python3` depending on your installation. The client requires the `numpy` library to be installed.]

Tips and Practical Considerations.

- To avoid overloading the server, we only allow each client to connect once every 30 seconds.
- The input file for python client should be organized such that the first n lines of the input file contain the signal components $x_1, \ldots, x_n$.
- For testing your code locally, use the code snippet below to simulate the channel (or an equivalent version in your preferred language). However, be sure to also check your code with the server using the python client as mentioned above as the demonstration on May 29th must use the server.

[1] https://www.epfl.ch/campus/services/en/it-services/network-services/remote-intranet-access/

Channel Model in Python.

The channel model is implemented on the server via the following function:

```
import numpy as np

def channel(x):
    x = np.asarray(x, dtype=float)

    if x.size % 2 != 0:
        raise ValueError(f"Input length must be even, got {x.size}.")

    i = int(np.random.randint(1, 5))   # uniform on {1,2,3,4}

    # Reshape into (n/2, 2) so each row is one complex symbol (a, b)
    pairs = x.reshape(-1, 2)
    a, b = pairs[:, 0], pairs[:, 1]

    if i == 1:
        Tx = np.stack([ a,  b], axis=1)
    elif i == 2:
        Tx = np.stack([-b,  a], axis=1)
    elif i == 3:
        Tx = np.stack([-a, -b], axis=1)
    else:  # i == 4
        Tx = np.stack([ b, -a], axis=1)

    Tx = Tx.reshape(-1)
    z  = np.random.standard_normal(Tx.shape)

    return (Tx + z)
```